DATA 260 - HOMEWORK 1

Grocery Supply and Recall Notices

Name - Vedant Vilas Vartak

SID4 = 3539

PORT_BASE = 8839

PREFIX = s3539

SEED = 3539

VERIFY_SEED = 263539

DOMAIN_ID = 3 - Grocery Supply and Recall Notices

Hardware - Apple M4 MacBook Air, 10 CPU cores, 16 GB memory

Local Model - qwen3:8b

AWS Region - us-east-2

GitHub Repository - https://github.com/vedantvartakwork/data260-3539

Tagged Commit - hw1

Part 1 - HTML, JavaScript, and Deployment

My assigned domain was grocery supply and recall notices because my DOMAIN_ID is 3. I created a form that allows someone to report a grocery product that may need to be recalled.

The form collects the product name, brand or manufacturer, submitter email, recall details and product category. The four categories are Produce, Meat and Seafood, Dairy and Refrigerated and Packaged Foods. The terms checkbox must also be selected before submission.

Code listing 1A - DOMAIN_SCHEMA.md (schema created before implementation)

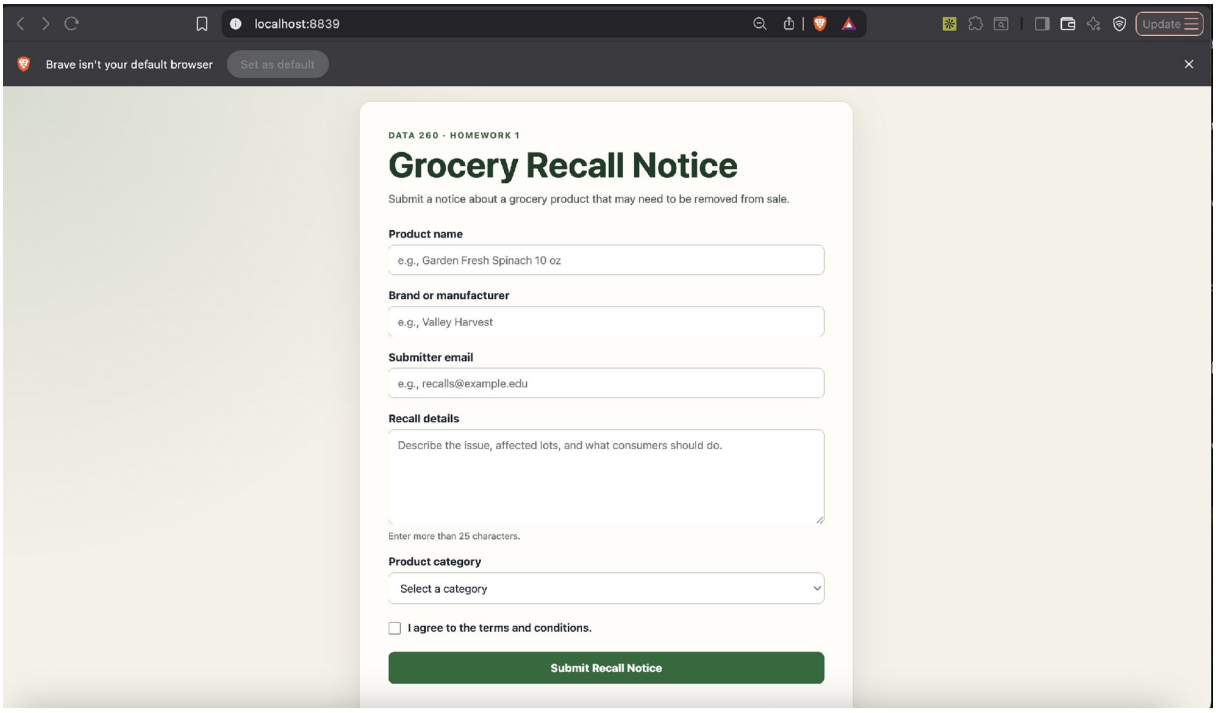
```
# Domain Schema

## Assigned domain

Grocery supply and recall notices (DOMAIN_ID 3)

## Entity

The form records a **grocery recall notice**.

| Field | Form control | Required | Description |
| --- | --- | --- | --- |
| `productName` | Text input | Yes | Name of the recalled grocery product |
| `brandName` | Text input | Yes | Brand or manufacturer name |
| `submitterEmail` | Email input | Yes | Email of the person submitting the notice |
| `recallDetails` | Textarea | Yes | Description of the issue, affected lots, and consumer guidance |
| `category` | Select | Yes | Grocery category for the affected product |
| `termsAccepted` | Checkbox | Yes | Confirms agreement to the terms and conditions |

## Category values

1. Produce
2. Meat and Seafood
3. Dairy and Refrigerated
4. Packaged Foods

## Example input

- Product name: Garden Fresh Spinach 10 oz
- Brand name: Valley Harvest
- Submitter email: recalls@example.edu
- Category: Produce
- Recall details: Selected bags may contain undeclared almonds. Consumers with nut allergies should not eat the product and should return it.

```

Code listing 1B - index.html (complete form)

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>HW1-Vedant Vilas Vartak</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <main class="card">
    <p class="eyebrow">DATA 260 · Homework 1</p>
    <h1>Grocery Recall Notice</h1>
    <p class="intro">Submit a notice about a grocery product that may need to be removed from sale.</p>

    <form id="recallForm" novalidate>
      <div class="field">
        <label for="productName">Product name</label>
        <input id="productName" name="productName" type="text"
               placeholder="e.g., Garden Fresh Spinach 10 oz" required autofocus>
      </div>

      <div class="field">
        <label for="brandName">Brand or manufacturer</label>
        <input id="brandName" name="brandName" type="text"
               placeholder="e.g., Valley Harvest" required>
      </div>

      <div class="field">
        <label for="submitterEmail">Submitter email</label>
        <input id="submitterEmail" name="submitterEmail" type="email"
               placeholder="e.g., recalls@example.edu" required>
      </div>

      <div class="field">
        <label for="recallDetails">Recall details</label>
        <textarea id="recallDetails" name="recallDetails" rows="6"
```

```
            placeholder="Describe the issue, affected lots, and what consumers should do."
            required></textarea>
      <small>Enter more than 25 characters.</small>
    </div>

    <div class="field">
      <label for="category">Product category</label>
      <select id="category" name="category" required>
        <option value="" selected disabled>Select a category</option>
        <option value="Produce">Produce</option>
        <option value="Meat and Seafood">Meat and Seafood</option>
        <option value="Dairy and Refrigerated">Dairy and Refrigerated</option>
        <option value="Packaged Foods">Packaged Foods</option>
      </select>
    </div>

    <div class="terms-row">
      <input id="termsAccepted" name="termsAccepted" type="checkbox">
      <label for="termsAccepted">I agree to the terms and conditions.</label>
    </div>

    <button type="submit">Submit Recall Notice</button>
    <p id="statusMessage" class="status" role="status" aria-live="polite"></p>
  </form>
 </main>

 <script src="app.js"></script>
</body>
</html>
```

Output 1 - Grocery recall form running at localhost:8839



JavaScript validation and successful submission

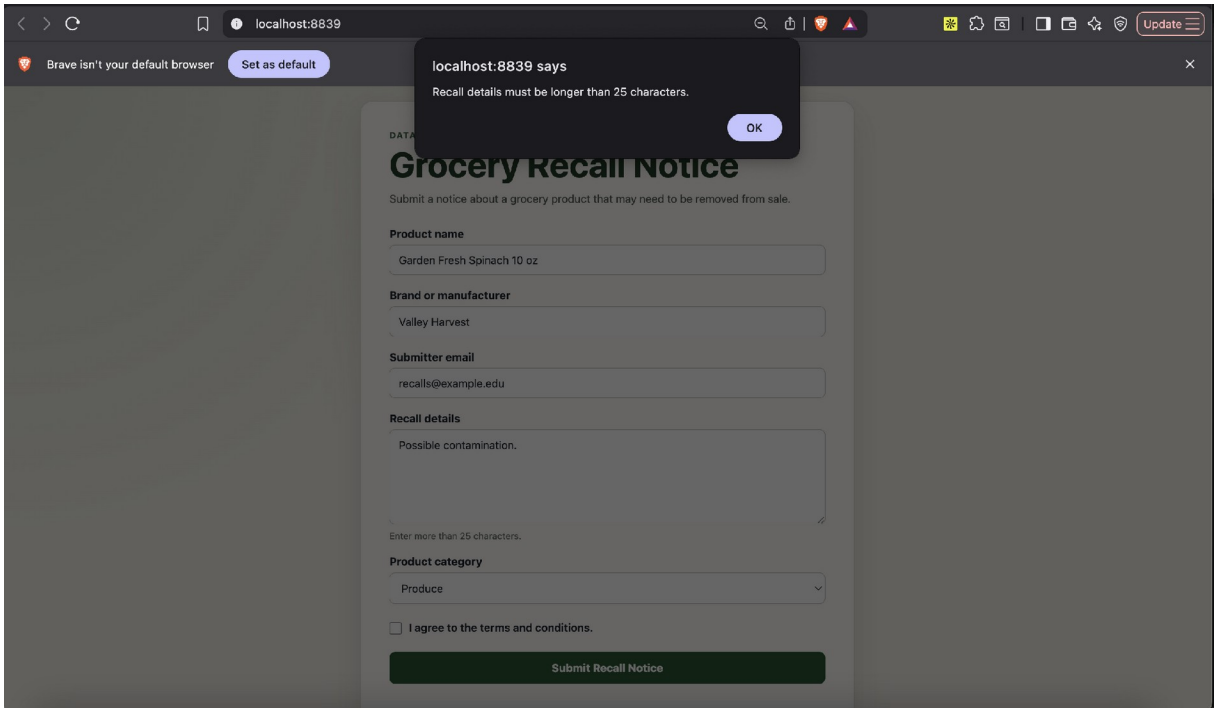
I used an arrow function to check that the recall description contains more than 25 characters and that the terms checkbox is selected. After a valid submission, the code creates and parses JSON, destructures the product name and email, adds submissionDate with the spread operator and increments a closure-based counter.

Code listing 2 - app.js (validation, JSON, destructuring, spread, and closure)

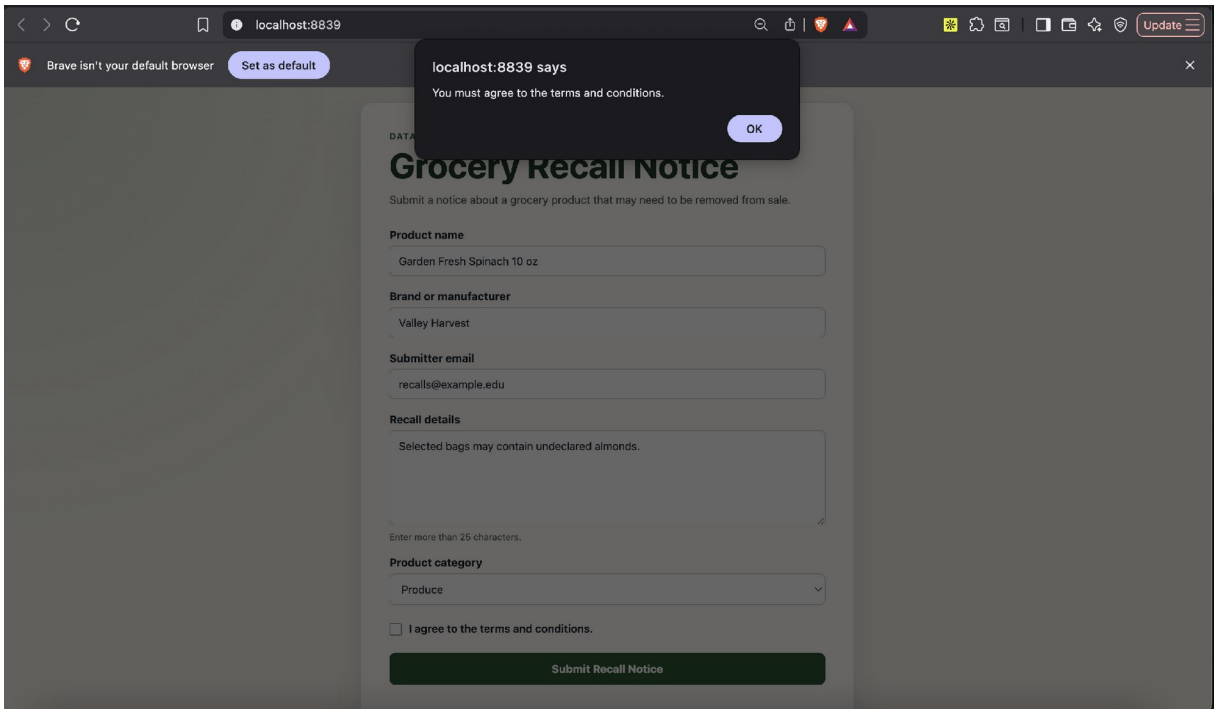
```
"use strict";

```

```
const createSubmissionCounter = () => {
  let count = 0;
  return () => {
    count += 1;
    return count;
  };
};

const countSuccessfulSubmission = createSubmissionCounter();

const validateForm = () => {
  const details = document.getElementById("recallDetails").value.trim();
  const termsAccepted = document.getElementById("termsAccepted").checked;

  if (details.length <= 25) {
    alert("Recall details must be longer than 25 characters.");
    return false;
  }

  if (!termsAccepted) {
    alert("You must agree to the terms and conditions.");
    return false;
  }

  return true;
};

document.getElementById("recallForm").addEventListener("submit", (event) => {
  event.preventDefault();

  const form = event.currentTarget;
  if (!form.checkValidity()) {
    form.reportValidity();
    return;
  }

  if (!validateForm()) return;

  const formData = new FormData(form);
  const submission = Object.fromEntries(formData.entries());
  submission.termsAccepted = document.getElementById("termsAccepted").checked;

  const jsonString = JSON.stringify(submission);
  console.log("Submission JSON:", jsonString);

  const parsedSubmission = JSON.parse(jsonString);
  const { productName, submitterEmail } = parsedSubmission;
  console.log("Product name:", productName);
  console.log("Submitter email:", submitterEmail);

  const updatedSubmission = {
    ...parsedSubmission,
    submissionDate: new Date().toISOString(),
  };
  console.log("Updated submission:", updatedSubmission);

  const submissionCount = countSuccessfulSubmission();
  console.log("Successful submission count:", submissionCount);
  document.getElementById("statusMessage").textContent =
    `Recall notice submitted successfully. Submission #${submissionCount}.`;
});
```

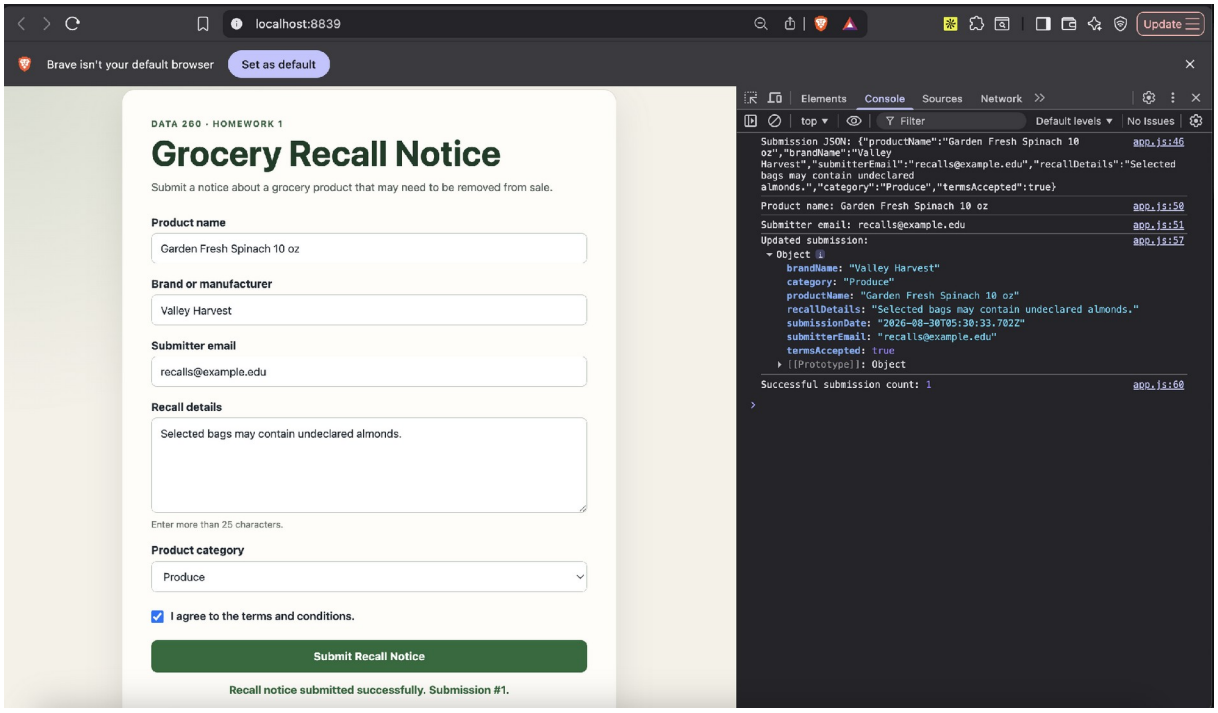
Output 2A - Alert for recall details containing 25 or fewer characters



Output 2B - Alert when the terms checkbox is not selected



Output 3 - Successful form submission and required browser-console values



Docker Deployment

I packaged the static application in an Nginx Docker image. The container listens on my assigned port 8839. I confirmed that the page returned HTTP status 200 and that Docker reported the container as healthy.

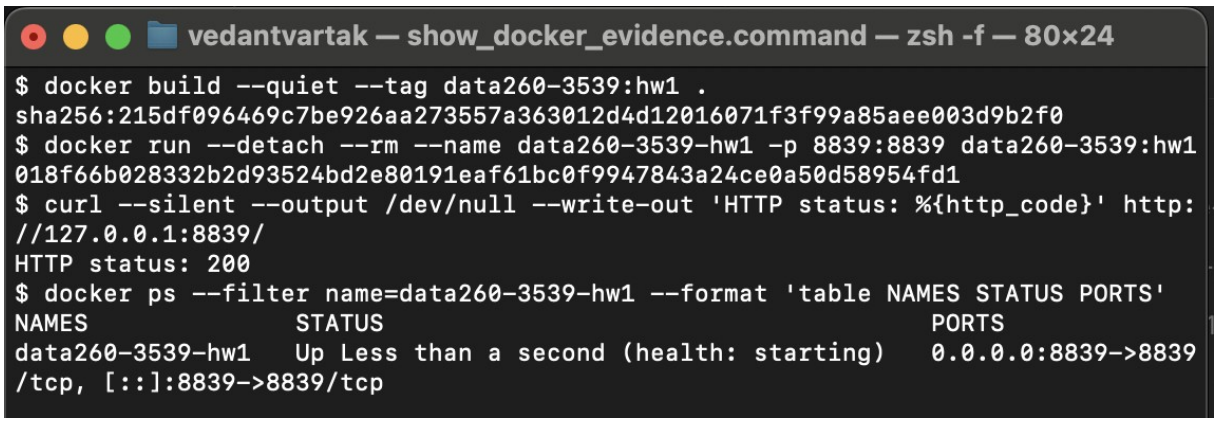
Code listing 3A - Dockerfile

```
FROM nginx:1.27-alpine

RUN rm -f /etc/nginx/conf.d/default.conf
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY index.html styles.css app.js /usr/share/nginx/html/

EXPOSE 8839

HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD wget -qO- http://127.0.0.1:8839/ >/dev/null || exit 1

```

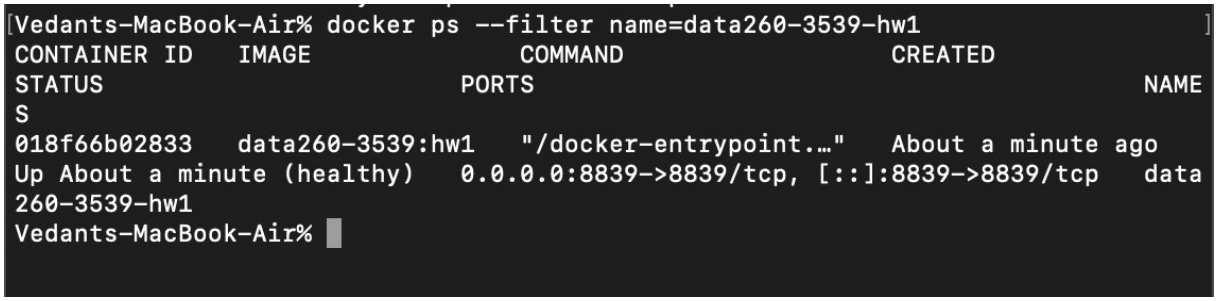
Code listing 3B - nginx.conf

```
server {
    listen 8839;
    listen [::]:8839;
    server_name _;

    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}

```

Output 4A - Docker build, container start, and HTTP 200 check



Output 4B - Docker container in healthy state



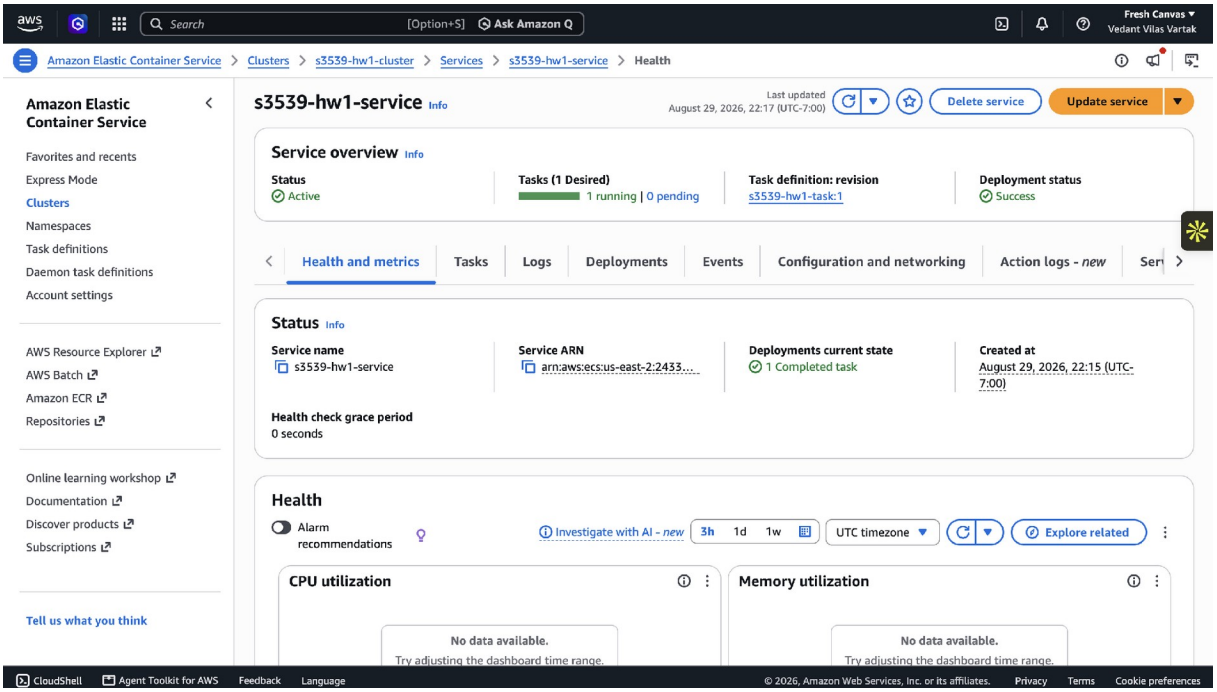
AWS ECS Deployment

I pushed the Docker image to a private Amazon ECR repository and deployed it with Amazon ECS Fargate in us-east-2. The service ran exactly one task. I opened the application through its public IP address and confirmed that it returned HTTP status 200. After collecting evidence, the homework resources were removed to avoid continued usage charges.

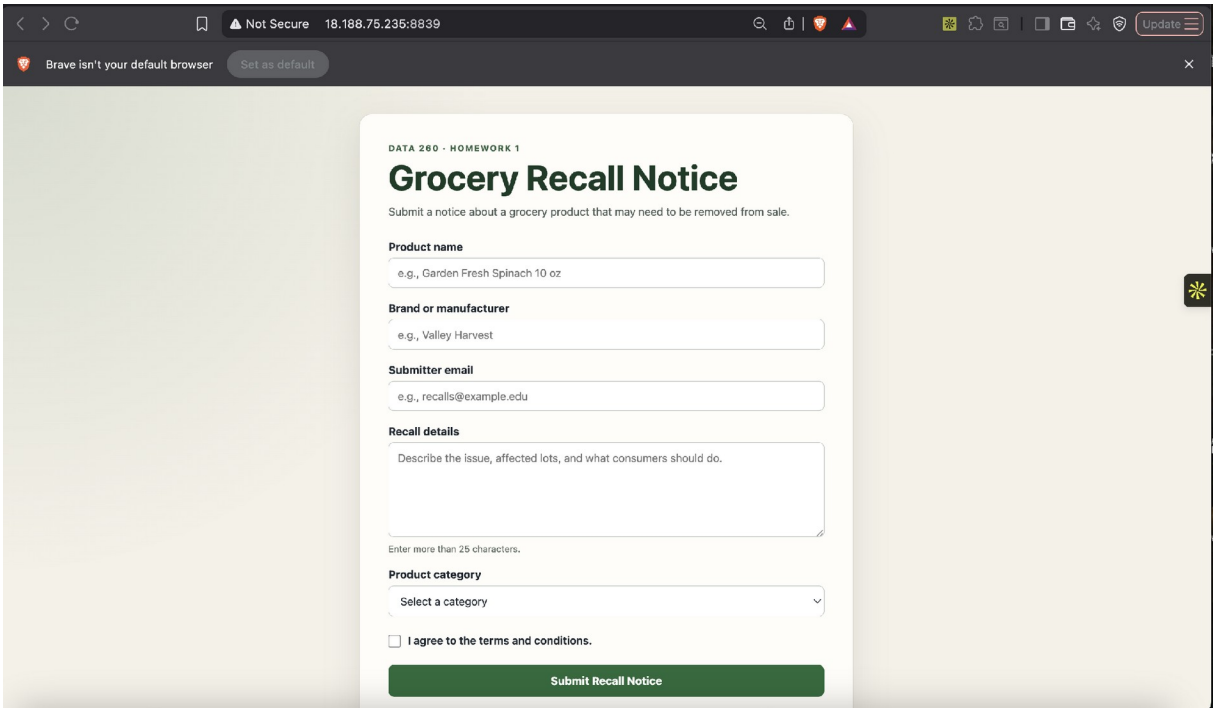
Code listing 4 - ECR push, one-task ECS service, and public-IP verification

```
  "${AWS[@]}" ecr describe-repositories --repository-names "$ECR_REPOSITORY" >/dev/null 2>&1 || \
    "${AWS[@]}" ecr create-repository --repository-name "$ECR_REPOSITORY" --image-scanning-configuration scanOnPush=true >/dev/null

  "${AWS[@]}" ecr get-login-password | \
    docker login --username AWS --password-stdin "$ACCOUNT_ID.dkr.ecr.$HW1_REGION.amazonaws.com"
  docker build --platform linux/amd64 -t "$ECR_REPOSITORY:latest" .
  docker tag "$ECR_REPOSITORY:latest" "$IMAGE_URI"
  docker push "$IMAGE_URI"
  ...
  TASK_DEFINITION_ARN="$("${AWS[@]}" ecs register-task-definition \
    --cli-input-json "file://$TASK_FILE" --query 'taskDefinition.taskDefinitionArn' --output text)"


NETWORK_CONFIGURATION="awsvpcConfiguration={subnets=[$SUBNETS],securityGroups=[$SECURITY_GROUP_ID],assignPublicIp=ENABLED}"
  SERVICE_STATUS="$("${AWS[@]}" ecs describe-services --cluster "$CLUSTER_NAME" --services "$SERVICE_NAME" --query 'services[0].status'
--output text)"
  if [[ "$SERVICE_STATUS" == "ACTIVE" ]]; then
    "${AWS[@]}" ecs update-service \
      --cluster "$CLUSTER_NAME" --service "$SERVICE_NAME" \
      --task-definition "$TASK_DEFINITION_ARN" --desired-count 1 \
      --network-configuration "$NETWORK_CONFIGURATION" >/dev/null
  else
    "${AWS[@]}" ecs create-service \
      --cluster "$CLUSTER_NAME" --service-name "$SERVICE_NAME" \
      --task-definition "$TASK_DEFINITION_ARN" --desired-count 1 \
      --launch-type FARGATE --platform-version LATEST \
      --network-configuration "$NETWORK_CONFIGURATION" >/dev/null
  fi

  "${AWS[@]}" ecs wait services-stable --cluster "$CLUSTER_NAME" --services "$SERVICE_NAME"
  TASK_ARN="$("${AWS[@]}" ecs list-tasks --cluster "$CLUSTER_NAME" --service-name "$SERVICE_NAME" --query 'taskArns[0]' --output text)"
  ENI_ID="$("${AWS[@]}" ecs describe-tasks --cluster "$CLUSTER_NAME" --tasks "$TASK_ARN" \
    --query 'tasks[0].attachments[0].details[?name==`networkInterfaceId`].value' --output text)"
  PUBLIC_IP="$("${AWS[@]}" ec2 describe-network-interfaces --network-interface-ids "$ENI_ID" \
    --query 'NetworkInterfaces[0].Association.PublicIp' --output text)"

  echo "ECS service is stable with desired count 1."
  echo "Public URL: http://$PUBLIC_IP:$HW1_PORT"
  curl --fail --retry 8 --retry-delay 5 "http://$PUBLIC_IP:$HW1_PORT/" >/dev/null
```

Output 5A - ECS service showing one desired and one running task



Output 5B - Application running on the AWS public IP address



Part 2 - Agentic AI: Planner, Reviewer, and Finalizer

Explanation:

Planner - The Planner reads the supplied grocery-recall title and content. It proposes exactly three topical tags and a one-sentence summary containing no more than 25 words.

Reviewer - The Reviewer checks whether the tags are distinct, specific, and supported by the input. It also checks the summary for factual accuracy, sentence count, and the 25-word limit, correcting the result when necessary.

Finalizer - The Finalizer validates the reviewed values and publishes only the three tags and summary as strict JSON. Reviewer notes are not included in the final publish object.

Command used - Planner/Reviewer/Finalizer demonstration

```
python3.12 agents_demo.py --input-file reports/hw01/cases/nondeterminism_input.json --model qwen3:8b --temperature 0.0
```

Code listing 5 - Complete validation and Planner/Reviewer/Finalizer implementation

```
def validate_publish(value: Any) -> dict[str, Any]:
    if not isinstance(value, dict):
        raise PipelineValidationError("output must be a JSON object")
    if set(value) != {"tags", "summary"}:
        raise PipelineValidationError("output must contain only tags and summary")

    tags = value.get("tags")
    summary = value.get("summary")
    if not isinstance(tags, list) or len(tags) != 3:
        raise PipelineValidationError("tags must contain exactly three strings")
    if not all(isinstance(tag, str) and len(tag.strip()) >= 2 for tag in tags):
        raise PipelineValidationError("each tag must be a meaningful non-empty string")
    normalized = [normalize_tag(tag) for tag in tags]
    if len(set(normalized)) != 3:
        raise PipelineValidationError("tags must not be duplicates")
    if any(tag in {"tag", "general", "information", "miscellaneous"} for tag in normalized):
        raise PipelineValidationError("tags must be topical rather than generic")

    if not isinstance(summary, str) or not summary.strip():
        raise PipelineValidationError("summary must be a non-empty string")
    summary = " ".join(summary.split())
    if word_count(summary) > 25:
        raise PipelineValidationError("summary must contain no more than 25 words")
    if len(re.findall(r"[.!?](?:[\"']|$)", summary)) != 1:
        raise PipelineValidationError("summary must be one sentence")

    return {"tags": [tag.strip() for tag in tags], "summary": summary}


def extract_json_object(text: str) -> dict[str, Any]:
    cleaned = text.strip().replace("```json", "").replace("```", "").strip()
    try:
        value = json.loads(cleaned)
    except json.JSONDecodeError:
        start = cleaned.find("{")
        end = cleaned.rfind("}")
        if start < 0 or end <= start:
            raise PipelineValidationError("response did not contain a JSON object")
        try:
            value = json.loads(cleaned[start : end + 1])
        except json.JSONDecodeError as exc:
            raise PipelineValidationError(f"malformed JSON: {exc.msg}") from exc
    if not isinstance(value, dict):
        raise PipelineValidationError("JSON response must be an object")
    return value


def request_valid_json(
    client: OllamaClient,
    messages: list[dict[str, str]],
    validator: Callable[[Any], dict[str, Any]],
    temperature: float,
) -> tuple[dict[str, Any], list[CompletionResult]]:
    attempts: list[CompletionResult] = []
```

```
    current_messages = list(messages)
    last_error = "unknown validation error"
    for attempt in range(2):
        result = client.complete(current_messages, temperature=temperature, json_mode=True)
        attempts.append(result)
        try:
            return validator(extract_json_object(result.content)), attempts
        except PipelineValidationError as exc:
            last_error = str(exc)
            if attempt == 0:
                current_messages = current_messages + [
                    {"role": "assistant", "content": result.content},
                    {
                        "role": "user",
                        "content": (
                            f"The response failed validation: {last_error}. "
                            "Return one corrected JSON object only."
                        ),
                    },
                ]
    raise PipelineValidationError(f"model output remained invalid after one retry: {last_error}")


def validate_review(value: Any) -> dict[str, Any]:
    if not isinstance(value, dict):
        raise PipelineValidationError("review must be an object")
    publish = validate_publish({"tags": value.get("tags"), "summary": value.get("summary")})
    notes = value.get("notes", [])
    if not isinstance(notes, list) or not all(isinstance(note, str) for note in notes):
        raise PipelineValidationError("review notes must be an array of strings")
    return {**publish, "notes": notes}


def run_pipeline(
    title: str,
    content: str,
    *,
    model: str = "qwen3:8b",
    temperature: float = 0.0,
    client: OllamaClient | None = None,
) -> dict[str, Any]:
    client = client or OllamaClient(model=model, temperature=temperature)
    source = f"Title: {title}\nContent: {content}"

    planner_messages = [
        {
            "role": "system",
            "content": (
                "You are the Planner. Derive topical labels and a concise summary only from the supplied title and content. "
                "Return valid JSON with exactly two keys: tags and summary. tags must be three distinct meaningful strings. "
                "summary must be one sentence of no more than 25 words. Do not use markdown."
            ),
        },
        {"role": "user", "content": source},
    ]
    planner, planner_calls = request_valid_json(client, planner_messages, validate_publish, temperature)

    reviewer_messages = [
        {
            "role": "system",
            "content": (
                "You are the Reviewer. Check whether the proposed tags are distinct, specific, and supported by the original input. "
                "Check that the summary is factual, one sentence, and at most 25 words. Correct problems when needed. "
                "Return only JSON with tags, summary, and notes. tags must contain exactly three strings; notes must be an array of short strings."
            ),
        },
        {
            "role": "user",
            "content": f"Original input:\n{source}\n\nPlanner proposal:\n{json.dumps(planner, ensure_ascii=False)}",
        },
    ]
    reviewer, reviewer_calls = request_valid_json(client, reviewer_messages, validate_review, temperature)

    final_publish = validate_publish({"tags": reviewer["tags"], "summary": reviewer["summary"]})
    changed = (
        [normalize_tag(tag) for tag in planner["tags"]] != [normalize_tag(tag) for tag in final_publish["tags"]]
        or planner["summary"].strip() != final_publish["summary"].strip()
    )

    def totals(calls: list[CompletionResult]) -> tuple[int, int]:
```

```
171         return sum(call.input_tokens for call in calls), sum(call.output_tokens for call in calls)
172
173     planner_in, planner_out = totals(planner_calls)
174     reviewer_in, reviewer_out = totals(reviewer_calls)
175     transcript = [
176         asdict(StageRecord("Planner", planner, planner_in, planner_out)),
177         asdict(StageRecord("Reviewer", {**reviewer, "changed": changed}, reviewer_in, reviewer_out)),
178         asdict(StageRecord("Finalizer", final_publish, 0, 0)),
179     ]
180     return {
181         "model": model,
182         "temperature": temperature,
183         "title": title,
184         "content": content,
185         "reviewer_changed": changed,
186         "transcript": transcript,
187         "publish": final_publish,
188         "token_usage": {
189             "input_tokens": planner_in + reviewer_in,
190             "output_tokens": planner_out + reviewer_out,
```

Output 6 - Exact command, Planner output, Reviewer output, Finalizer output, and final publish JSON

```
$ python3.12 agents_demo.py --input-file reports/hw01/cases/nondeterminism_input.json --model qwen3:8b --temperature 0.0

--- Planner ---
{
  "tags": [
    "food recall",
    "almond allergy",
    "spinach contamination"
  ],
  "summary": "Valley Harvest recalls spinach bags with undeclared almonds, affecting lot code VH0826."
}

--- Reviewer ---
{
  "tags": [
    "food recall",
    "almond allergy",
    "spinach contamination"
  ],
  "summary": "Valley Harvest recalls spinach bags with undeclared almonds, affecting lot code VH0826.",
  "notes": [
    "Tags are distinct, specific, and supported by the content.",
    "Summary is factual, concise, and within the word limit."
  ],
  "changed": false
}

--- Finalizer ---
{
  "tags": [
    "food recall",
    "almond allergy",
    "spinach contamination"
  ],
  "summary": "Valley Harvest recalls spinach bags with undeclared almonds, affecting lot code VH0826."
}

--- Final Publish JSON ---
{
  "tags": [
    "food recall",
    "almond allergy",
    "spinach contamination"
  ],
  "summary": "Valley Harvest recalls spinach bags with undeclared almonds, affecting lot code VH0826."
```

Short answers:

Q1 : The final tags were food recall, almond allergy, and spinach contamination.

Q2: Valley Harvest recalls spinach bags with undeclared almonds, affecting lot code VH0826.

Q3: No. The Reviewer kept the planner's result because the tags were distinct and supported by the input and the summary was factual, concise and within the word limit.

Part 3 - Measuring Non-Determinism

I saved one grocery-recall input and used it unchanged for all 40 runs. The pipeline ran 20 times at temperature 0.7 and 20 times at temperature 0.0. All 40 runs completed successfully.

Code listing 6A - Fixed input used unchanged for all 40 trials

```
{
  "title": "Undeclared almond recall for Valley Harvest spinach",
  "content": "Valley Harvest is recalling selected 10-ounce bags of Garden Fresh Spinach because the packages may contain undeclared almonds. The affected bags have lot code VH0826 and a best-by date of September 2, 2026. Customers with almond allergies should not eat the product and may return it to the store for a refund."
}
```

Code listing 6B - Running and saving the 40 trials

```
    parser.add_argument("--runs", type=int, default=20)
    parser.add_argument("--model", default="qwen3:8b")
    parser.add_argument("--raw-dir", type=Path, default=Path("reports/hw01/raw"))
    args = parser.parse_args()

    title, content = load_input(args.input)
    args.raw_dir.mkdir(parents=True, exist_ok=True)
    results: list[dict[str, Any]] = []
    log_lines = [f"[{timestamp()}] Started non-determinism experiment: model={args.model}, runs_per_temperature={args.runs}"]

    for temperature in (0.7, 0.0):
        for run_number in range(1, args.runs + 1):
            started = time.perf_counter()
            record: dict[str, Any] = {
                "temperature": temperature,
                "run_number": run_number,
                "model": args.model,
                "started_at": timestamp(),
            }
            try:
                pipeline = run_pipeline(title, content, model=args.model, temperature=temperature)
                record.update(
                    {
                        "success": True,
                        "tags": pipeline["publish"]["tags"],
                        "summary": pipeline["publish"]["summary"],
                        "reviewer_changed": pipeline["reviewer_changed"],
                        "token_usage": pipeline["token_usage"],
                        "error": None,
                    }
                )
            except Exception as exc:  # preserve every failed run for auditability
                record.update({"success": False, "tags": [], "summary": "", "error": f"{type(exc).__name__}: {exc}"})
            record["latency_ms"] = round((time.perf_counter() - started) * 1000, 2)
            record["finished_at"] = timestamp()
            results.append(record)
            status = "success" if record["success"] else "failure"
            line = f"[{record['finished_at']}] temp={temperature:.1f} run={run_number:02d} {status} latency_ms={record['latency_ms']:.2f}"
            log_lines.append(line)
            print(line, flush=True)

    json_path = args.raw_dir / "nondeterminism_results.json"
    csv_path = args.raw_dir / "nondeterminism_results.csv"
    json_path.write_text(json.dumps(results, indent=2, ensure_ascii=False) + "\n", encoding="utf-8")
    with csv_path.open("w", encoding="utf-8", newline="") as handle:
        writer = csv.DictWriter(
            handle,
            fieldnames=["temperature", "run_number", "model", "success", "tags", "summary", "latency_ms", "error"],
            extrasaction="ignore",
        )
        writer.writeheader()
        for result in results:
            writer.writerow({**result, "tags": json.dumps(result["tags"], ensure_ascii=False)})

    log_lines.append(f"[{timestamp()}] Finished experiment with {sum(bool(item['success']) for item in results)}/{len(results)} successful runs")
    Path("reports/hw01/RUN_LOG.txt").write_text("\n".join(log_lines) + "\n", encoding="utf-8")
    subprocess.run(
        [sys.executable, "scripts/generate_metrics.py", "--input", str(json_path), "--output", "reports/hw01/METRICS.md"],
        check=True,
    )
```

Code listing 6C - Distinct tag sets, tag frequencies, and latency percentiles

```
def normalized_tag_set(tags: list[str]) -> tuple[str, ...]:
    return tuple(sorted(normalize_tag(tag) for tag in tags))


def nearest_rank(values: list[float], percentile: float) -> float:
    if not values:
        raise ValueError("cannot calculate a percentile without values")
    ordered = sorted(values)
    index = max(0, math.ceil(percentile * len(ordered)) - 1)
    return ordered[index]


def summarize(runs: list[dict[str, Any]], temperature: float) -> dict[str, Any]:
    successful = [run for run in runs if run.get("success") and float(run["temperature"]) == temperature]
    tag_sets = {normalized_tag_set(run["tags"]) for run in successful}
    tag_counts: Counter[str] = Counter()
    for run in successful:
        tag_counts.update(set(normalized_tag_set(run["tags"])))
    latencies = [float(run["latency_ms"]) for run in successful]
    return {
        "successful_runs": len(successful),
        "distinct_tag_sets": len(tag_sets),
        "tags_all": sorted(tag for tag, count in tag_counts.items() if count == len(successful)) if successful else [],
        "tags_once": sorted(tag for tag, count in tag_counts.items() if count == 1),
        "p50": nearest_rank(latencies, 0.50) if latencies else None,
        "p95": nearest_rank(latencies, 0.95) if latencies else None,
        "p99": nearest_rank(latencies, 0.99) if latencies else None,
    }


def display_list(values: list[str]) -> str:
    return ", ".join(f"`{value}`" for value in values) if values else "None"


def render_metrics(runs: list[dict[str, Any]]) -> str:
    summaries = {temperature: summarize(runs, temperature) for temperature in (0.7, 0.0)}
    lines = [
        "# Homework 1 Metrics",
        "",
        "Generated from `reports/hw01/raw/nondeterminism_results.json`.",
        "",
        "## Non-determinism",
        "",
        "Tag comparison is case-insensitive, ignores leading/trailing and repeated whitespace, and treats tag order as irrelevant.",
        "",
        "| Metric | Temp 0.7 | Temp 0.0 |",
        "| --- | ---: | ---: |",
        f"| Successful runs | {summaries[0.7]['successful_runs']} | {summaries[0.0]['successful_runs']} |",
        f"| Distinct tag sets | {summaries[0.7]['distinct_tag_sets']} | {summaries[0.0]['distinct_tag_sets']} |",
        f"| Tags in all successful runs | {display_list(summaries[0.7]['tags_all'])} | {display_list(summaries[0.0]['tags_all'])} |",
        f"| Tags appearing once | {display_list(summaries[0.7]['tags_once'])} | {display_list(summaries[0.0]['tags_once'])} |",
        "",
        "| Latency metric | Temp 0.7 | Temp 0.0 |",
        "| --- | ---: | ---: |",
    ]
    for key, label in (("p50", "p50"), ("p95", "p95"), ("p99", "p99")):
        left = "N/A" if summaries[0.7][key] is None else f"{summaries[0.7][key]:.2f} ms"
        right = "N/A" if summaries[0.0][key] is None else f"{summaries[0.0][key]:.2f} ms"
        lines.append(f"| {label} | {left} | {right} |")
    lines.extend(
        [
            "",
            "Percentiles use the nearest-rank method: sort successful latency values and select rank `ceil(p * n)`.",
            "Failed runs are retained in the raw file but excluded from tag and latency calculations.",
            "",
        ]
    )
    return "\n".join(lines)
```

Output 7 - Completion of 40 runs and generated metrics

```
Vedants-MacBook-Air% ./scripts/show_metrics_evidence.command

[$ grep 'Finished experiment' reports/hw01/RUN_LOG.txt
[2026-08-26T06:12:30.594236+00:00] Finished experiment with 40/40 successful runs

$ sed -n '9,22p' reports/hw01/METRICS.md
| Metric | Temp 0.7 | Temp 0.0 |
| --- | ---: | ---: |
| Successful runs | 20 | 20 |
| Distinct tag sets | 3 | 1 |
| Tags in all successful runs | `almond allergy`, `food recall` | `almond allergy`, `food recall`, `spinach contamination` |
| Tags appearing once | None | None |

| Latency metric | Temp 0.7 | Temp 0.0 |
| --- | ---: | ---: |
| p50 | 9879.65 ms | 11041.02 ms |
| p95 | 10180.85 ms | 11199.12 ms |
| p99 | 10442.98 ms | 11285.88 ms |

Percentiles use the nearest-rank method: sort successful latency values and select rank `ceil(p * n)`.

Metrics evidence is ready. Keep this window open for the screenshot.
Vedants-MacBook-Air%
```

Measured results

Metric	Temperature 0.7	Temperature 0.0
Successful runs	20	20
Distinct tag sets	3	1
Tags in all runs	almond allergy; food recall	almond allergy; food recall; spinach contamination
Tags appearing once	None	None
Latency p50	9879.65 ms	11041.02 ms
Latency p95	10180.85 ms	11199.12 ms
Latency p99	10442.98 ms	11285.88 ms

Interpretation using my results

Identical users: At temperature 0.7, two users sending identical input could receive slightly different but related tag combinations. My runs produced three distinct tag sets. At temperature 0.0, all 20 runs produced the same tag set so this setting was more consistent for this input and model.

Acceptable vs. unacceptable variation: Variation is acceptable for brainstorming possible labels because several relevant alternatives may be useful. It is not acceptable when model output directly controls a safety decision, such as whether a contaminated product requires a recall, because that decision must be consistent and verified.

Part 4 - Model Client and Token Accounting

All model calls use the reusable adapter in src/model_client.py. The interactive client loads AGENT.md as its system instruction, prints token usage after every model response and implements /stats without changing the history.

Code listing 7A - Stable Ollama model adapter

```
@dataclass(frozen=True)
class CompletionResult:
    content: str
    input_tokens: int
    output_tokens: int
    model: str
    total_duration_ns: int

    @property
    def total_tokens(self) -> int:
        return self.input_tokens + self.output_tokens


class ModelClientError(RuntimeError):
    """Raised when Ollama cannot return a usable response."""


class OllamaClient:
    def __init__(
        self,
        model: str | None = None,
        base_url: str | None = None,
        temperature: float = 0.0,
        timeout: float = 180.0,
    ) -> None:
        self.model = model or os.environ.get("OLLAMA_MODEL", "qwen3:8b")
        self.base_url = (base_url or os.environ.get("OLLAMA_URL", "http://127.0.0.1:11434")).rstrip("/")
        self.temperature = temperature
        self.timeout = timeout

    def complete(
        self,
        messages: list[dict[str, str]],
        tools: list[dict[str, Any]] | None = None,
        *,
        temperature: float | None = None,
        json_mode: bool = False,
    ) -> CompletionResult:
        """Complete a chat turn through a stable project-level interface."""
        payload: dict[str, Any] = {
            "model": self.model,
            "messages": messages,
            "stream": False,
            "think": False,
            "options": {
                "temperature": self.temperature if temperature is None else temperature,
                "num_ctx": 4096,
                "num_predict": 320,
            },
        }
        if os.environ.get("OLLAMA_SEED"):
            payload["options"]["seed"] = int(os.environ["OLLAMA_SEED"])
        if tools:
            payload["tools"] = tools
        if json_mode:
            payload["format"] = "json"

        request = urllib.request.Request(
            f"{self.base_url}/api/chat",
            data=json.dumps(payload).encode("utf-8"),
            headers={"Content-Type": "application/json"},
            method="POST",
        )
        try:
            with urllib.request.urlopen(request, timeout=self.timeout) as response:
                body = json.load(response)
        except (urllib.error.URLError, TimeoutError, json.JSONDecodeError) as exc:
            raise ModelClientError(f"Ollama request failed: {exc}") from exc

```

```
        try:
            return CompletionResult(
                content=str(body["message"]["content"]),
                input_tokens=int(body.get("prompt_eval_count", 0)),
                output_tokens=int(body.get("eval_count", 0)),
                model=str(body.get("model", self.model)),
                total_duration_ns=int(body.get("total_duration", 0)),
            )
        except (KeyError, TypeError, ValueError) as exc:
            raise ModelClientError("Ollama returned an unexpected response shape") from exc


def complete(
    messages: list[dict[str, str]],
    tools: list[dict[str, Any]] | None = None,
) -> CompletionResult:
    """Convenience interface using the default client configuration."""
    return OllamaClient().complete(messages, tools=tools)
```

Code listing 7B - AGENT.md bullet-only review instructions

```
# Code Review Instructions

- Respond using bullet points only.
- Do not write an introductory or closing paragraph.
- Begin each bullet with one of: `BUG:`, `RISK:`, `STYLE:`, or `OK:`.
- Cite a line number when the supplied code includes line numbers.
- Explain the concrete effect of every reported issue.
- Do not invent problems; use one `OK:` bullet when no actionable issue exists.

```

Code listing 7C - Token accounting, history, /stats, and final totals

```
@dataclass
class SessionStats:
    turns: int = 0
    input_tokens: int = 0
    output_tokens: int = 0


class ReviewSession:
    def __init__(self, client: OllamaClient, instructions: str) -> None:
        self.client = client
        self.history: list[dict[str, str]] = [{"role": "system", "content": instructions}]
        self.stats = SessionStats()

    def serialized_history_length(self) -> int:
        return len(json.dumps(self.history, ensure_ascii=False, separators=(",", ":")))

    def stats_snapshot(self) -> dict[str, int]:
        return {
            "turn_count": self.stats.turns,
            "cumulative_input_tokens": self.stats.input_tokens,
            "cumulative_output_tokens": self.stats.output_tokens,
            "serialized_history_length": self.serialized_history_length(),
        }

    def submit(self, prompt: str) -> tuple[str, dict[str, int]]:
        messages = self.history + [{"role": "user", "content": prompt}]
        result = self.client.complete(messages)
        self.history.extend(
            [
                {"role": "user", "content": prompt},
                {"role": "assistant", "content": result.content},
            ]
        )
        self.stats.turns += 1
        self.stats.input_tokens += result.input_tokens
        self.stats.output_tokens += result.output_tokens
        usage = {
            "input_tokens": result.input_tokens,
            "output_tokens": result.output_tokens,
            "total_tokens": result.total_tokens,
        }
        return result.content, usage


def print_stats(snapshot: dict[str, int]) -> None:
    print("Session statistics:")
    print(f"  Turn count: {snapshot['turn_count']}")
    print(f"  Cumulative input tokens: {snapshot['cumulative_input_tokens']}")
```

```
    print(f"  Cumulative output tokens: {snapshot['cumulative_output_tokens']}")
    print(f"  Serialized conversation-history length: {snapshot['serialized_history_length']} characters")


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--model", default="qwen3:8b")
    parser.add_argument("--agent-file", type=Path, default=Path(__file__).with_name("AGENT.md"))
    args = parser.parse_args()

    instructions = args.agent_file.read_text(encoding="utf-8")
    session = ReviewSession(OllamaClient(model=args.model), instructions)
    print("Bullet-only code reviewer. Enter code or a review request. Commands: /stats, /quit")

    try:
        while True:
            prompt = input("review> ").strip()
            if not prompt:
                continue
            if prompt == "/stats":
                print_stats(session.stats_snapshot())
                continue
            if prompt in {"/quit", "/exit"}:
                break
            response, usage = session.submit(prompt)
            print(response)
            print(
                f"Tokens - input: {usage['input_tokens']}, output: {usage['output_tokens']}, "
                f"total: {usage['total_tokens']}"
            )
    finally:
        print("\nFinal totals:")
        print(f"  Cumulative input tokens: {session.stats.input_tokens}")
        print(f"  Cumulative output tokens: {session.stats.output_tokens}")
        print(f"  Turn count: {session.stats.turns}")


if __name__ == "__main__":
    main()
```

Output 8A - Per-turn tokens and /stats after turn 3

```
Vedants-MacBook-Air% python3.12 scripts/show_token_evidence.py --part 1
Saved model: qwen3:8b
Model turns: 5; /stats commands: 2

Turn 1
Prompt: Review this JavaScript: const total = prices.reduce((sum, price) => sum + price, 0);
- OK: The code correctly calculates the total sum of the `prices` array using `reduce`.
Tokens: input=133, output=21, total=154

Turn 2
Prompt: Review this validation: if (details.length < 25) alert('Too short');
- RISK: The alert is not user-friendly; consider using a more accessible method like a modal or toast notification.
- STYLE: The comment `/no_think` is not relevant in the code and should be removed.
- STYLE: The condition could be improved by using a more descriptive variable name instead of `details`.
- STYLE: Missing error handling for cases where `details` might not be a string or array.
Tokens: input=181, output=89, total=270

Turn 3
Prompt: Review this Python: tags = list(set(tags))[:3]
- STYLE: The line `/no_think` is not relevant in the code and should be removed.
- STYLE: Using `set` on `tags` will lose order, consider using `sorted(set(tags))[:3]` if order matters.
- STYLE: The variable name `tags` is reused after conversion, which could lead to confusion.
- OK: The code correctly limits the result to the first 3 unique tags.
Tokens: input=292, output=92, total=384
/stats after turn 3: {"turn_count": 3, "cumulative_input_tokens": 606, "cumulative_output_tokens": 202, "serialized_history_length": 1666}

Vedants-MacBook-Air%
```

Output 8B - Per-turn tokens, /stats after turn 5, and cumulative totals

```
Vedants-MacBook-Air% python3.12 scripts/show_token_evidence.py --part 2
Saved model: qwen3:8b
Model turns: 5; /stats commands: 2

Turn 4
Prompt: Review this shell command: docker run -p 8839:8839 data260-3539
- BUG: The command lacks a command to execute inside the container, which will result in an error as the container will not know what process
- STYLE: The port mapping `-p 8839:8839` is redundant as both ports are the same; consider removing it for clarity.
- STYLE: The image name `data260-3539` should be checked for correctness and availability in a registry.
Tokens: input=421, output=96, total=517

Turn 5
Prompt: Review this JSON parsing: data = json.loads(response)
- STYLE: The line `/no_think` is not relevant in the code and should be removed.
- STYLE: Consider adding error handling for potential JSON parsing errors using a try-except block.
- OK: The code correctly parses the JSON response into a Python dictionary or list.
Tokens: input=537, output=60, total=597
/stats after turn 5: {"turn_count": 5, "cumulative_input_tokens": 1564, "cumulative_output_tokens": 358, "serialized_history_length": 2554}

Final totals: {"turn_count": 5, "cumulative_input_tokens": 1564, "cumulative_output_tokens": 358, "serialized_history_length": 2554}
Vedants-MacBook-Air%
```

Recorded token statistics

Observation	Turns	Input	Output	History length
/stats after turn 3	3	606	202	1666
/stats after turn 5	5	1564	358	2554

Conceptual answers:

Why is prior context resent?

A chat model does not automatically remember separate API calls. The application resends earlier messages so the model has the context needed to understand the current request.

System prompt versus user message.

A system prompt defines the model's overall role, rules, and response style and has higher priority. A user message contains the individual request being made during the conversation.

Why do input tokens grow?

Input tokens increase because every new request contains the latest message plus more of the previous conversation. The model must process that growing history again during each turn.

What limits the growth?

Growth is eventually limited by the model's maximum context window. When the history approaches that limit, older material must be removed, summarized or moved into a new conversation.

AI-Use Disclosure:

1. What I used an AI assistant for and what I did myself

I used ChatGPT to understand the homework requirements, get hints when I was stuck and improve the speed of my workflow. I completed the brainstorming, implementation, testing, verification and final work myself.

2. One unsuitable AI output or independently verified item

At one point, ChatGPT provided a general answer based on assumptions that did not exactly match my project. I did not use that answer directly. I compared it with my actual work and corrected the information myself.

3. How I detected or verified it

I reviewed the entire assignment again and compared each requirement with my work and results. I also pasted some results into ChatGPT to check whether they appeared expected, but I independently verified everything against the assignment requirements.

4. What I changed and why it works now

I did not automatically apply recommendations from ChatGPT. I treated its responses as suggestions rather than ground truth and made changes only after checking them myself. I used AI to make my workflow faster, not to complete the work for me.